

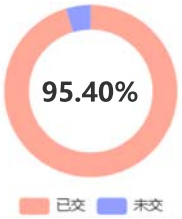
计算机系统基础

课程信息
班级管理
课程资料
教学计划
练习考试
作答分析
作业
测验
考试
统计分析
签到
证书管理
笔记
讨论区
课程评价
公告
MOOC申请

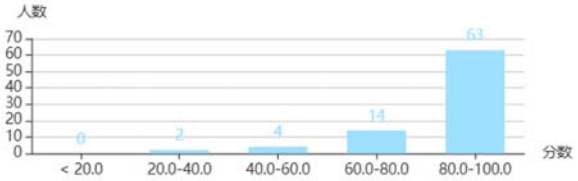
3.9 越界访问和缓冲区溢出

返回

题数：4 总分：100 有效时间：2024-10-31 08:30:00/2024-11-17 08:20:00 已交/总人数：83/87







作业详情 作业分析

[默认选中为正确答案]

(一) 单选题

1，以下关于缓冲区溢出以及缓冲区溢出攻击的叙述中，错误的是（）

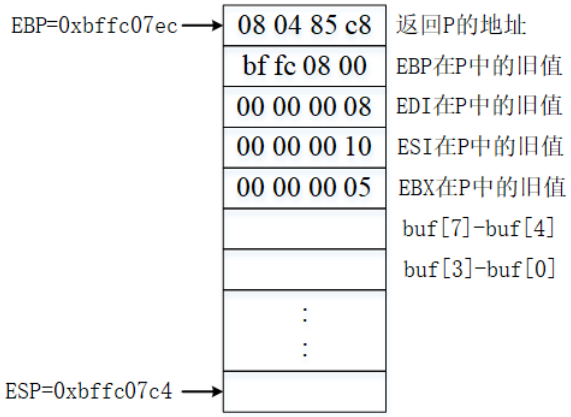
○	A.当传送到栈中的局部数组中的字符的个数超过数组长度时发生缓冲区溢出	0.00%，0人
○	B.恶意程序可利用像strcpy等无字符串长度设定的c库函数进行缓冲区溢出攻击	0.00%，0人
○	C.只要发生缓冲区溢出，寄存器内容或变量或返回地址等程序信息就可能被修改	0.00%，0人
○	D.只要发生缓冲区溢出，CPU就会跳转到恶意程序事先设定好的程序去执行	100.00%，83人

2，以下给出函数getline存在漏洞和问题的C代码实现与对应的反汇编部分结果。

```
char *getline( )
{
   char buf[8];
   char *result;
   gets(buf);
   result = malloc(strlen(buf));
   strcpy(result, buf);
   return result;
}
 1 0804840c      <getline>:
 2 0804840c:     55              push  %ebp
 3 0804840d:     89 e5           mov   %esp,  %ebp
 4 0804840f:     83 ec 28        sub   $0x28,  %esp
 5 08048412:     89 5d f4        mov   %ebx,  -0xc(%ebp)
 6 08048415:     89 75 f8        mov   %esi,   -0x8(%ebp)
 7 08048418:     89 7d fc        mov   %edi,  -0x4(%ebp)
 8 0804841b:     8d 75 ec        lea    -0x14(%ebp),  %esi
 9 0804841e:     89 34 24        mov   %esi,   (%esp)
10 08048421:     e8 a3 ff ff ff  call 80483c9 <gets>
```

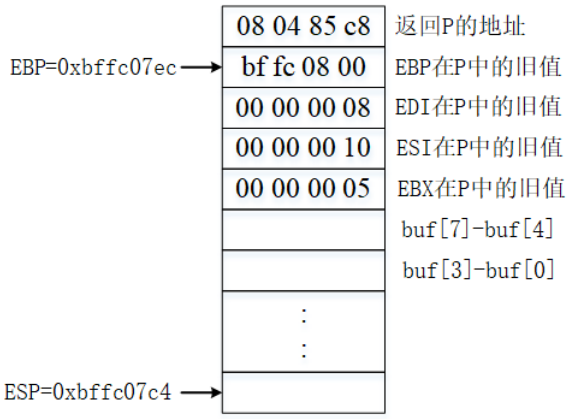
假定有一个调用过程P调用了函数getline，其返回地址为0x80485c8，为调用getline函数而执行完call指令时，部分寄存器的内容如下：R[ebp]=0xbffc0800,R[esp]=0xbffc07f0,R[ebx]=0x5,R[esi]=0x10,R[edi]=0x8。
执行程序时从标准输入读入的一行字符串为“0123456789ABCDEF0123456789\0”，此时程序发生段错误(segmentation fault)并中止执行，经调试确认错误是在执行getline的ret指令时发生的。
请问执行完第7行指令后，栈中信息存放情况为（ ）

A.



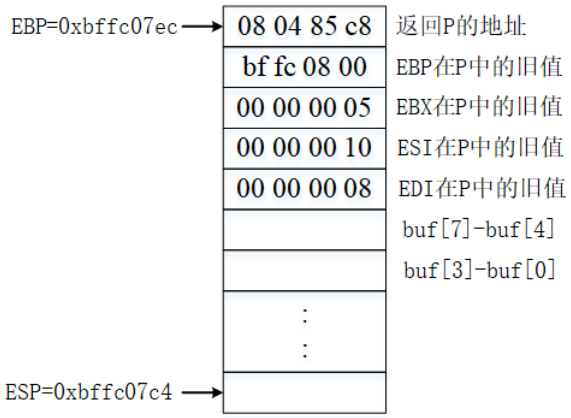
8.43%，7人

B.



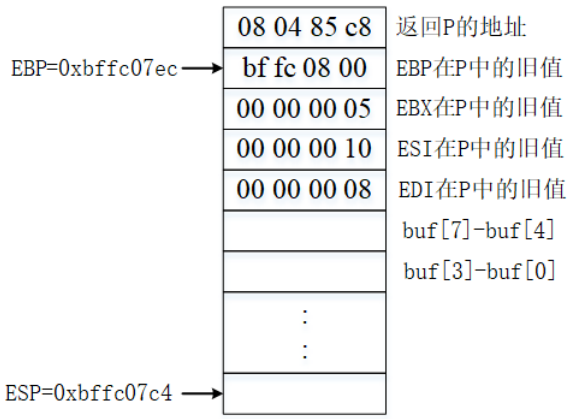
87.95%，73人

C.



1.20%，1人

D.



2.41%，2人

3，以下给出函数getline存在漏洞和问题的C代码实现与对应的反汇编部分结果。

```
char *getline( )
{
   char buf[8];
   char *result;
   gets(buf);
   result = malloc(strlen(buf));
   strcpy(result, buf);
   return result;
}
```

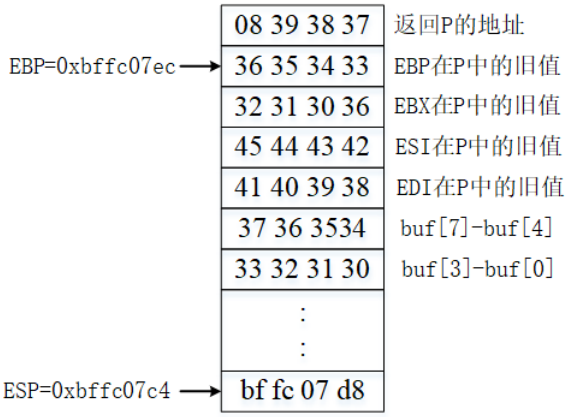
```
 1  0804840c        <getline>:
 2  0804840c:       55                  push  %ebp
 3  0804840d:       89  e5              mov   %esp,  %ebp
 4  0804840f:       83  ec  28          sub   $0x28,  %esp
 5  08048412:       89  5d  f4          mov   %ebx,  -0xc(%ebp)
 6  08048415:       89  75  f8          mov   %esi,   -0x8(%ebp)
 7  08048418:       89  7d  fc          mov   %edi,  -0x4(%ebp)
 8  0804841b:       8d  75  ec          lea     -0x14(%ebp),  %esi
 9  0804841e:       89  34  24          mov   %esi,    (%esp)
10  08048421:       e8 a3 ff ff ff      call 80483c9 <gets>
```

假定有一个调用过程P调用了函数getline，其返回地址为0x80485c8，为调用getline函数而执行完call指令时，部分寄存器的内容如下：R[ebp]=0xbffc0800,R[esp]=0xbffc07f0,R[ebx]=0x5,R[esi]=0x10,R[edi]=0x8。

执行程序时从标准输入读入的一行字符串为“0123456789ABCDEF0123456789\0”，此时程序发生段错误(segmentation fault)并中止执行，经调试确认错误是在执行getline的ret指令时发生的。

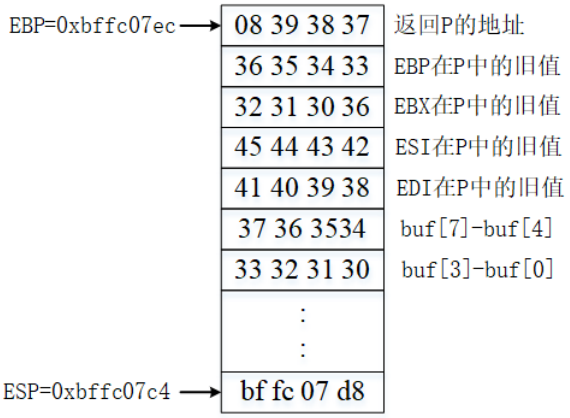
请问执行完第10行指令并调用gets函数后，回到第10的下一条指令执行前，栈中信息存放情况为（ ）

A.



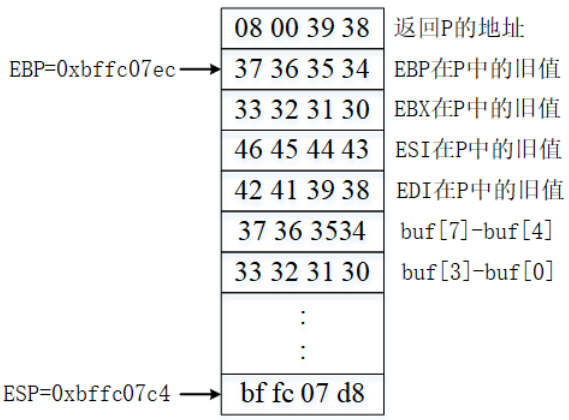
2.41% , 2人

B.



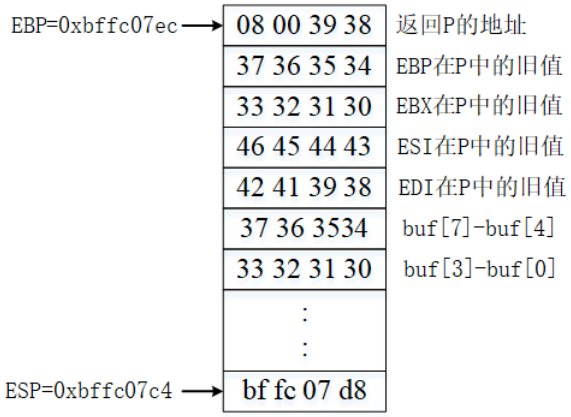
4.82% , 4人

C.



91.57% , 76人

D.



1.20% , 1人

4，以下给出函数getline存在漏洞和问题的C代码实现与对应的反汇编部分结果。

```
char *getline( )
{
   char buf[8];
   char *result;
   gets(buf);
   result = malloc(strlen(buf));
   strcpy(result, buf);
   return result;
}
```

```
 1 0804840c      <getline>:
 2 0804840c:    55              push  %ebp
 3 0804840d:    89  e5          mov   %esp,  %ebp
 4 0804840f:    83  ec  28      sub   $0x28,  %esp
 5 08048412:    89  5d  f4      mov   %ebx,  -0xc(%ebp)
 6 08048415:    89  75  f8      mov   %esi,  -0x8(%ebp)
 7 08048418:    89  7d  fc      mov   %edi,  -0x4(%ebp)
 8 0804841b:    8d  75  ec      lea    -0x14(%ebp),  %esi
 9 0804841e:    89  34  24      mov   %esi,   (%esp)
10 08048421:    e8 a3 ff ff ff  call 80483c9 <gets>
```

假定有一个调用过程P调用了函数getline，其返回地址为0x80485c8，为调用getline函数而执行完call指令时，部分寄存器的内容如下：R[ebp]=0xbffc0800,R[esp]=0xbffc07f0,R[ebx]=0x5,R[esi]=0x10,R[edi]=0x8。
执行程序时从标准输入读入的一行字符串为“0123456789ABCDEF0123456789\0”，此时程序发生段错误(segmentation fault)并中止执行，经调试确认错误是在执行getline的ret指令时发生的。
请问当执行到getline的ret指令时，假如程序不发生段错误，则正确的返回地址是什么？发生段错误是因为执行getline的ret指令时得到了什么样的返回地址？

A.假如不发生错误，正确返回地址是0x8003938，执行getline的ret指令时得到的返回地址是0x8003938

4.82% , 4人

B.假如不发生错误，正确返回地址是0x80485c8，执行getline的ret指令时得到的返回地址是0x80485c8

4.82% , 4人

C.假如不发生错误，正确返回地址是0x80485c8，执行getline的ret指令时得到的返回地址是0x8393837

3.61% , 3人

D.假如不发生错误，正确返回地址是0x80485c8，执行getline的ret指令时得到的返回地址是0x8003938

86.75% , 72人

Copyright © 2016-2025 浙江省高等学校在线开放课程共享平台　　浙ICP备05018780号-10　　浙公网安备 33010602010741号
关注在浙学微信公众号